

UNIVERSIDADE DO OESTE DE SANTA CATARINA
CURSO DE SISTEMAS DE INFORMAÇÃO
DESENVOLVIMENTO PARA DISPOSITIVOS MÓVEIS

JORGE LUIZ DO NASCIMENTO

JCONDO MOBILE

Projeto de aplicativo mobile para gestão e comunicação condominial

Trabalho apresentado ao componente curricular Desenvolvimento para Dispositivos Móveis, referente à Atividade Avaliativa 1, sob orientação do professor da disciplina.

CHAPECÓ
2026

SUMÁRIO

1	INTRODUÇÃO
2	REQUISITOS DO APLICATIVO
2.1	Requisitos funcionais
2.2	Requisitos não funcionais
2.3	Regras de negócio
3	ESCOLHA DA PLATAFORMA
3.1	Alternativas avaliadas
3.2	Justificativa da escolha
3.3	Tecnologias previstas
4	ARQUITETURA E VIABILIDADE TÉCNICA
4.1	Organização em camadas
4.2	Modelo de dados
4.3	Recursos do dispositivo e consumo
5	PROTÓTIPO
5.1	Diretrizes de interface
5.2	Identidade visual
5.3	Fluxo de navegação
5.4	Mockups das telas
5.5	Descrição das telas
6	MVP, EVOLUÇÃO E RISCOS
7	CONCLUSÃO
	REFERÊNCIAS

1 INTRODUÇÃO

Moro em um condomínio de porte médio e boa parte da comunicação entre moradores e administração ainda acontece por caminhos improvisados. Aviso de manutenção sai em papel colado no elevador. Reserva do salão de festas é anotada num caderno na portaria. Reclamação de lâmpada queimada vira mensagem em grupo de WhatsApp, se perde entre outras conversas e ninguém sabe se foi resolvida. Foi observando isso que escolhi o tema deste trabalho.

O JCondo Mobile é a proposta de um aplicativo que junta essas três coisas em um lugar só: comunicados, reservas de área comum e registro de ocorrências. O público-alvo são os moradores, o síndico e a administração do condomínio.

1.1 Problema

Sem um canal único, três problemas se repetem. O primeiro é o morador não ficar sabendo de um aviso importante, porque não passou pelo mural naquele dia. O segundo é o conflito de agenda: duas famílias reservam o salão para o mesmo sábado porque a anotação foi feita em folhas diferentes. O terceiro é a ocorrência sem rastro, aquela que o morador relatou mas não tem como cobrar depois, porque não existe protocolo nem status.

1.2 Objetivo geral

Projetar um aplicativo mobile de gestão condominial que centralize a comunicação e disponibilize os serviços básicos do condomínio de forma rápida, organizada e acessível pelo celular.

1.3 Objetivos específicos

- Centralizar os comunicados da administração em um único lugar, com data e prioridade.
- Permitir a reserva de áreas comuns com verificação automática de disponibilidade.
- Registrar ocorrências com número de protocolo e status de acompanhamento.
- Disponibilizar ao morador seus dados cadastrais e suas preferências.
- Notificar o morador quando houver um comunicado novo.
- Deixar a arquitetura preparada para receber pagamentos, documentos e assembleias digitais em versões futuras.

2 REQUISITOS DO APLICATIVO

Os requisitos foram separados em funcionais, que dizem o que o sistema precisa fazer, e não funcionais, que tratam de qualidade e restrições. Acrescentei também uma lista de regras de negócio, porque algumas decisões do condomínio não cabiam bem em nenhuma das duas categorias e são justamente as que definem o comportamento do aplicativo nos casos difíceis.

A coluna de prioridade usa dois valores. "Essencial" marca o que precisa estar na primeira versão para o aplicativo fazer sentido. "Desejável" marca o que agrega, mas pode ficar para depois sem inviabilizar o uso.

2.1 Requisitos funcionais

Tabela 1 - Requisitos funcionais

ID	Descrição	Prioridade
RF01	O aplicativo deve permitir que o morador se autentique informando e-mail e senha.	Essencial
RF02	O aplicativo deve manter a sessão do morador entre execuções, evitando pedir a senha a cada abertura.	Essencial
RF03	O aplicativo deve permitir encerrar a sessão a qualquer momento.	Essencial
RF04	A tela inicial deve apresentar uma visão resumida: saudação, avisos recentes, próxima reserva e quantidade de ocorrências abertas.	Essencial
RF05	A tela inicial deve oferecer atalhos diretos para nova reserva e nova ocorrência.	Desejável
RF06	O aplicativo deve listar os comunicados publicados pela administração, do mais recente para o mais antigo.	Essencial
RF07	Cada comunicado deve indicar sua prioridade (informativo, importante ou urgente) e permitir a leitura do conteúdo completo.	Essencial
RF08	O aplicativo deve listar as áreas comuns disponíveis para reserva, com capacidade, horário de funcionamento e regras de uso.	Essencial
RF09	O aplicativo deve exibir os horários livres e ocupados de uma área em uma data escolhida.	Essencial
RF10	O aplicativo deve permitir confirmar a reserva de um horário livre e apresentar a confirmação ao morador.	Essencial
RF11	O aplicativo deve listar as reservas do morador e permitir o cancelamento das que ainda não ocorreram.	Essencial
RF12	O aplicativo deve permitir o registro de uma ocorrência informando categoria, assunto e descrição.	Essencial
RF13	Ao registrar uma ocorrência, o aplicativo deve apresentar ao morador um número de protocolo.	Essencial
RF14	O aplicativo deve listar as ocorrências do morador com o status de cada uma e a resposta da administração, quando houver.	Essencial

ID	Descrição	Prioridade
RF15	O aplicativo deve exibir os dados cadastrais do morador e permitir a atualização de e-mail e telefone.	Essencial
RF16	O aplicativo deve notificar o morador quando um novo comunicado for publicado.	Desejável
RF17	O morador deve poder ligar e desligar as notificações de comunicado.	Desejável
RF18	A administração deve poder publicar comunicados.	Desejável
RF19	O aplicativo deve permitir anexar uma foto à ocorrência.	Desejável

Fonte: o autor (2026).

2.2 Requisitos não funcionais

Tabela 2 - Requisitos não funcionais

ID	Categoria	Descrição
RNF01	Usabilidade	A interface deve ser adequada ao uso em smartphone, com as funções principais alcançáveis em no máximo dois toques a partir da tela inicial.
RNF02	Usabilidade	Toda ação que altera dados deve dar retorno visível ao morador, indicando sucesso ou o motivo da falha.
RNF03	Desempenho	As telas principais devem carregar em até três segundos em condições normais de rede.
RNF04	Disponibilidade	Falhas de conectividade devem ser tratadas com mensagem clara e opção de tentar novamente, sem encerrar o aplicativo.
RNF05	Segurança	As senhas não podem ser armazenadas em texto puro em nenhum ponto do sistema.
RNF06	Segurança	O acesso aos dados deve respeitar o perfil do usuário: um morador só enxerga as próprias reservas e ocorrências.
RNF07	Responsividade	O layout deve se adaptar a diferentes tamanhos e densidades de tela sem corte de conteúdo.
RNF08	Acessibilidade	Os alvos de toque devem ter no mínimo 48dp e o contraste entre texto e fundo deve atender à razão 4,5:1.
RNF09	Acessibilidade	Nenhuma informação pode ser transmitida apenas por cor.
RNF10	Manutenção	O código deve ser modular e separado por responsabilidade, para permitir a inclusão de novas funcionalidades sem reescrita.
RNF11	Compatibilidade	A primeira versão deve rodar em Android 8.0 (API 26) ou superior.
RNF12	Persistência	Os dados devem ser armazenados em banco relacional no servidor, e não apenas no aparelho.
RNF13	Consumo	O aplicativo não deve manter sensores ativos em segundo plano nem coletar localização.

Fonte: o autor (2026).

2.3 Regras de negócio

Estas regras são do condomínio, não do software. Elas valem independentemente de o morador usar o aplicativo, o telefone ou a portaria, e por isso precisam ser garantidas pelo servidor.

Tabela 3 - Regras de negócio

ID	Regra
RN01	Uma área comum não pode ter duas reservas confirmadas que se sobreponham no tempo, na mesma data.
RN02	Reservas encostadas não são conflito: um horário que termina às 16h e outro que começa às 16h podem coexistir.
RN03	Não é permitido reservar data ou horário que já passou.
RN04	A reserva precisa caber inteiramente dentro do horário de funcionamento da área.
RN05	A reserva pode ser feita com no máximo 90 dias de antecedência.
RN06	Somente o próprio morador (ou a administração) pode cancelar uma reserva, e apenas antes de ela ocorrer.
RN07	Toda ocorrência recebe um protocolo único no momento da abertura.
RN08	Bloco, apartamento e CPF são dados cadastrais e só podem ser alterados pela administração.
RN09	Somente o perfil de administração pode publicar comunicados.

Fonte: o autor (2026).

3 ESCOLHA DA PLATAFORMA

3.1 Alternativas avaliadas

Considereei três caminhos: desenvolvimento nativo para Android, nativo para iOS e uma solução híbrida (React Native ou Flutter). O quadro abaixo resume o que pesou em cada critério.

Tabela 4 - Comparativo entre as plataformas avaliadas

Critério	Android nativo	iOS nativo	Híbrido
Alcance no público-alvo	Maioria dos moradores do condomínio usa Android, inclusive aparelhos de entrada.	Parcela menor do público, concentrada em faixas de renda mais altas.	Cobriria os dois, mas o alcance imediato seria o mesmo do Android.
Custo para desenvolver e testar	Emulador gratuito e teste em aparelho físico próprio.	Exige Mac e conta paga de desenvolvedor.	Emulador gratuito, mas uma camada a mais para configurar.
Aderência à disciplina	Android Studio, Java e Gradle são as ferramentas estudadas.	Swift e Xcode não fazem parte do conteúdo.	Exigiria aprender JavaScript ou Dart em paralelo.
Acesso a recursos do aparelho	Direto, pela API do sistema.	Direto, pela API do sistema.	Depende de plugin de terceiro para alguns recursos.
Documentação e material de apoio	Documentação oficial extensa e comunidade grande.	Documentação boa, comunidade menor em português.	Boa, mas mais volátil entre versões.
Risco para o prazo do trabalho	Baixo.	Alto: dependeria de equipamento que não tenho.	Médio: curva de aprendizado adicional.

Fonte: o autor (2026).

3.2 Justificativa da escolha

A plataforma escolhida é o Android nativo. O critério que mais pesou foi o alcance: o aplicativo é para os moradores deste condomínio, e a maior parte deles usa Android, incluindo aparelhos de entrada e modelos mais antigos. Um aplicativo de gestão condominial que não roda no celular do morador não resolve o problema que motivou o projeto.

O segundo critério foi a viabilidade dentro do prazo e dos recursos que tenho. Desenvolver para iOS exigiria um Mac e uma conta paga de desenvolvedor, o que inviabilizaria a entrega. A opção híbrida seria defensável em um projeto real que precisasse das duas plataformas desde o início, mas aqui ela acrescentaria uma

camada de abstração e uma linguagem nova sem trazer ganho, já que a segunda plataforma não é prioridade.

O terceiro critério foi o alinhamento com a disciplina. Android Studio, Java e Gradle são as ferramentas trabalhadas em aula, e usá-las permite aplicar o conteúdo estudado em vez de dividir o esforço com uma tecnologia paralela.

A escolha do Android nativo não fecha a porta para o iOS. Como a lógica de negócio fica no servidor e não no aplicativo, uma futura versão para iOS consumiria a mesma API, sem duplicar regras. Essa é a principal razão de eu ter desenhado a arquitetura em camadas do jeito que está descrito na seção 4.

Versão mínima suportada

Defini o Android 8.0 (API 26) como piso. Segundo os painéis de distribuição do Android Studio, essa versão cobre a quase totalidade dos aparelhos em uso, e permite usar recursos modernos da linguagem sem bibliotecas de compatibilidade. Descer para versões anteriores traria pouco alcance adicional e bastante trabalho extra.

3.3 Tecnologias previstas

Tabela 5 - Tecnologias previstas para o desenvolvimento

Componente	Tecnologia	Finalidade
Aplicativo	Android SDK, minSdk 26	Execução no smartphone do morador.
Linguagem	Java	Lógica de apresentação e integração com a API.
IDE e build	Android Studio e Gradle	Desenvolvimento, compilação, testes e depuração.
Interface	Material Design	Componentes com acessibilidade e comportamento de toque já resolvidos.
Cliente HTTP	Retrofit e Gson	Consumo dos endpoints REST e conversão do JSON.
Backend	Spring Boot, API REST	Autenticação, regras de negócio e persistência.
Banco de dados	MySQL	Armazenamento de moradores, avisos, áreas, reservas e ocorrências.
Autenticação	Sessão por token	Controle de acesso entre o aplicativo e a API.
Notificações	Canal de notificação do Android	Aviso ao morador sobre novos comunicados.
Documentação da API	Swagger / OpenAPI	Consulta e teste dos endpoints sem o aplicativo.

Fonte: o autor (2026).

4 ARQUITETURA E VIABILIDADE TÉCNICA

4.1 Organização em camadas

A solução foi organizada em três camadas. Essa separação não é enfeite de documento: ela é o que permite que a regra de conflito de reserva funcione de verdade e o que abre caminho para uma futura versão iOS sem reescrever nada além da interface.

Figura 1 - Arquitetura em três camadas



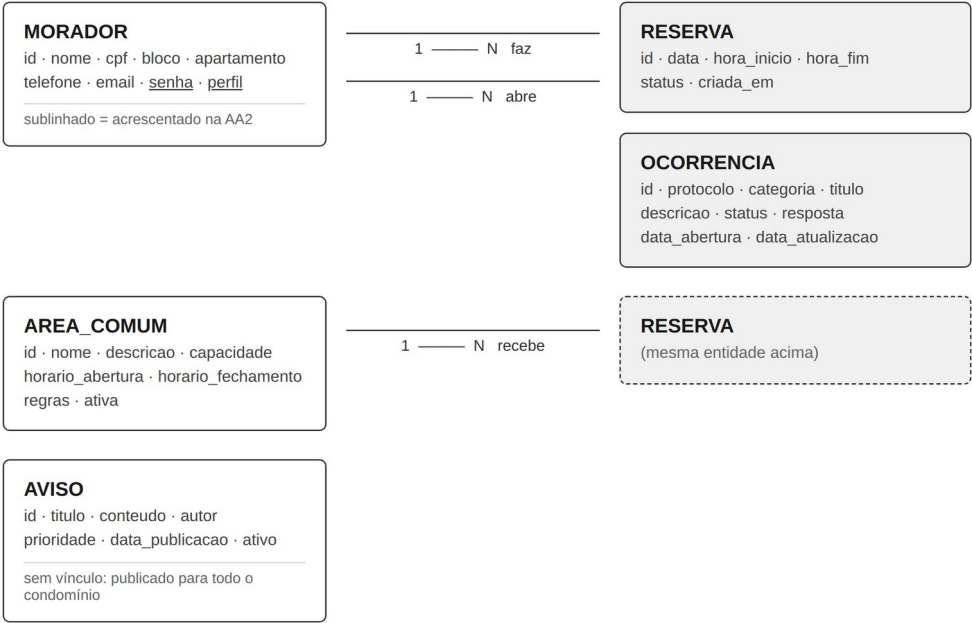
Fonte: o autor (2026).

A decisão mais importante desta arquitetura é que nenhuma regra de negócio vive apenas no aplicativo. O aplicativo pode antecipar validações para dar retorno rápido ao morador, escondendo horários ocupados, por exemplo, mas quem decide se uma reserva pode ser gravada é o servidor. Se a verificação ficasse só no celular, dois moradores que abrissem a tela ao mesmo tempo veriam o mesmo horário livre e os dois gravariam.

4.2 Modelo de dados

São cinco entidades. O morador é também o usuário do aplicativo, porque no condomínio todo usuário é um morador cadastrado, e manter dois cadastros separados criaria a necessidade de sincronizá-los sem benefício real.

Figura 2 - Modelo de entidades



Fonte: o autor (2026).

4.3 Recursos do dispositivo e consumo

Fiz questão de decidir cedo quais recursos do aparelho o aplicativo usa, porque cada permissão pedida é um motivo a mais para o morador desconfiar e desinstalar. O JCondo não precisa saber onde a pessoa está nem ficar ouvindo sensores.

Tabela 6 - Uso de recursos do aparelho

Recurso	Uso previsto	Cuidado adotado
Rede	Única fonte de dados: todas as telas consomem a API.	Tempo limite curto nas requisições, para avisar rápido em rede ruim.
Notificações	Alertar sobre novos comunicados.	Permissão pedida em tempo de execução e um interruptor no perfil para desligar.
Armazenamento local	Guardar o token da sessão e as preferências.	A senha não é armazenada em lugar nenhum do aparelho.
Câmera	Anexar foto à ocorrência (funcionalidade desejável).	Permissão pedida apenas no momento do uso, e não na instalação.
GPS	Não utilizado.	Evita consumo de bateria e coleta desnecessária de dados pessoais.
Biometria	Possível recurso futuro para o login.	Usaria o mecanismo do próprio sistema, sem armazenar dado biométrico.

Fonte: o autor (2026).

4.4 Viabilidade

O projeto é viável porque não depende de nada incomum. Autenticação, consumo de API REST, listagem, formulário e notificação local são recursos presentes em qualquer aplicativo mobile e amplamente documentados. O ponto que exige mais atenção é a regra de conflito de reserva, e mesmo ela se resolve com uma consulta ao banco, desde que a verificação aconteça no mesmo lugar onde o dado é gravado.

5 PROTÓTIPO

5.1 Diretrizes de interface

Antes de desenhar as telas, defini seis diretrizes. Elas serviram como critério para decidir o que entra em cada tela e o que fica de fora.

Tabela 7 - Diretrizes de interface adotadas

Diretriz	O que significa na prática
Hierarquia visual	A informação mais consultada aparece primeiro na tela. Na tela inicial, isso significa: quem é o morador, os números do condomínio, o compromisso mais próximo e só depois os atalhos.
Consistência	Cartão, botão e etiqueta se comportam igual em todas as telas. Uma etiqueta verde significa a mesma coisa em reservas e em ocorrências.
Feedback	Toda ação que grava algo termina com uma confirmação visível. Reserva confirmada mostra data e horário; ocorrência registrada mostra o protocolo.
Acessibilidade	Alvo de toque mínimo de 48dp, contraste adequado e informação nunca transmitida só por cor. A etiqueta de horário ocupado traz a palavra "Ocupado" junto do cinza.
Responsividade	Layouts com medidas relativas e listas roláveis, para funcionar de telas pequenas a tablets.
Redução de etapas	As funções frequentes ficam a um toque da tela inicial. A reserva completa exige três decisões e nada mais: área, data e horário.

Fonte: o autor (2026).

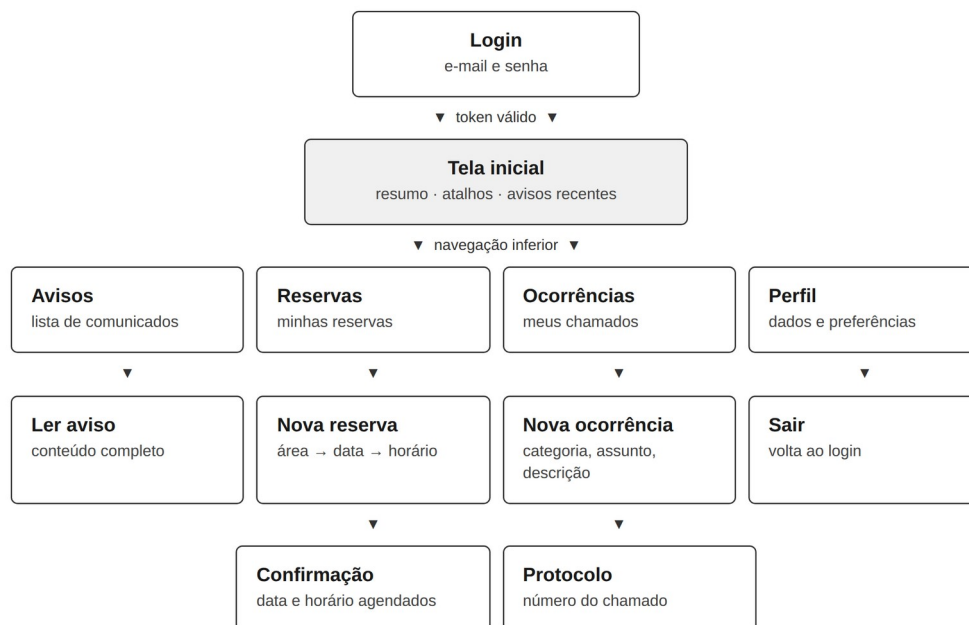
5.2 Identidade visual

A paleta usa azul escuro, branco e tons neutros de cinza. O azul escuro aparece no cabeçalho e nas ações principais, transmitindo a ideia de organização e de instituição. O branco domina o corpo das telas, o que reduz a poluição visual e faz os cartões de conteúdo se destacarem. As cores de estado (verde para confirmado, âmbar para em andamento, vermelho para urgente ou cancelado) aparecem só nas etiquetas, sempre acompanhadas de texto.

A navegação fica em uma barra inferior com cinco itens, dentro da área que o polegar alcança sem trocar a mão de posição. Cada item corresponde a uma das áreas do aplicativo.

5.3 Fluxo de navegação

Figura 3 - Fluxo de navegação entre as telas



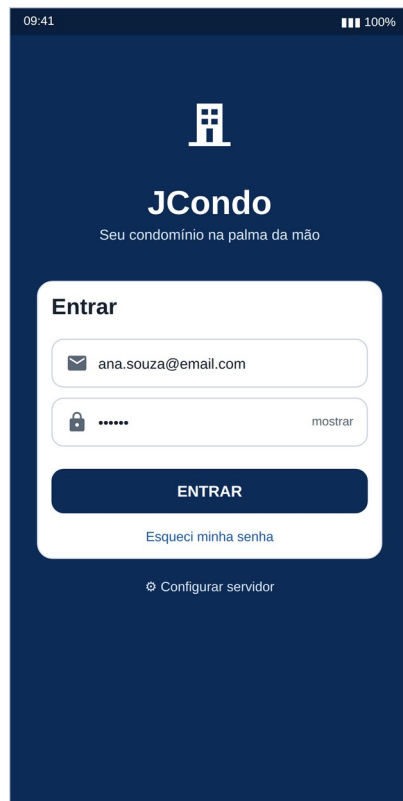
Fonte: o autor (2026).

O fluxo principal do morador é curto. Ele abre o aplicativo e, se já tiver entrado antes, cai direto na tela inicial. De lá vê o resumo, toca em um aviso para ler, ou usa um dos atalhos para reservar uma área ou abrir uma ocorrência. A barra inferior está sempre disponível para trocar de área.

5.4 Mockups das telas

Os mockups abaixo representam a proposta de interface. Foram desenhados na proporção real de um smartphone e já refletem as diretrizes descritas na seção 5.1 e a identidade visual da seção 5.2.

Figura 4 - Tela de login e tela inicial



Login

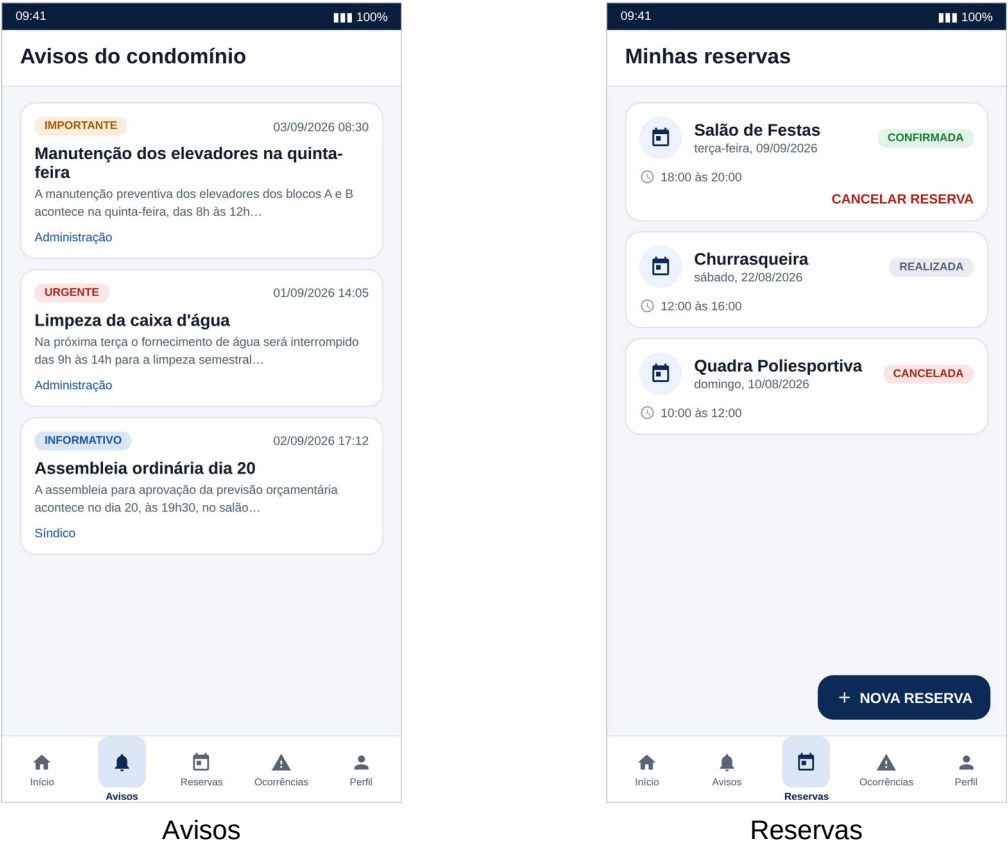


Início

Fonte: o autor (2026).

Na tela de login, o cartão branco sobe sobre o fundo azul para separar a identidade visual, no topo, da ação, no formulário. A tela inicial abre com uma saudação e a unidade do morador, seguida de três contadores que respondem de imediato às perguntas mais comuns: tem aviso novo, tenho reserva marcada, tenho ocorrência em aberto.

Figura 5 - Lista de avisos e lista de reservas



Fonte: o autor (2026).

A lista de avisos usa uma etiqueta com o texto da prioridade, e não apenas a cor, para atender à diretriz de acessibilidade. Cada card mostra as duas primeiras linhas do comunicado, o suficiente para o morador decidir se abre. Na lista de reservas, o botão de cancelar só aparece nas reservas confirmadas que ainda vão acontecer, em vez de aparecer desabilitado nas demais.

Figura 6 - Nova reserva e nova ocorrência

09:41

100%

←

Nova reserva

Área comum

Salão de Festas

▼

Espaço coberto para até 60 pessoas - funciona das 10:00 às 22:00

Data

📅

quinta-feira, 10/09/2026

Horários disponíveis

10:00 - 12:00

12:00 - 14:00

14:00 - 16:00

16:00 - 18:00

18:00 - 20:00

20:00 - 22:00

Ocupado

Ocupado

Ocupado

CONFIRMAR RESERVA

09:41

100%

←

Nova ocorrência

Iluminação

▼

Categoria

Lâmpada queimada na garagem

Ex.: Lâmpada queimada na garagem

27/120

A lâmpada da vaga 42, no subsolo 1, está queimada há três dias. À noite o corredor fica bem escuro e fica difícil enxergar quem circula por ali.

Descreva o local e o problema com o máximo de detalhes possível.

REGISTRAR OCORRÊNCIA

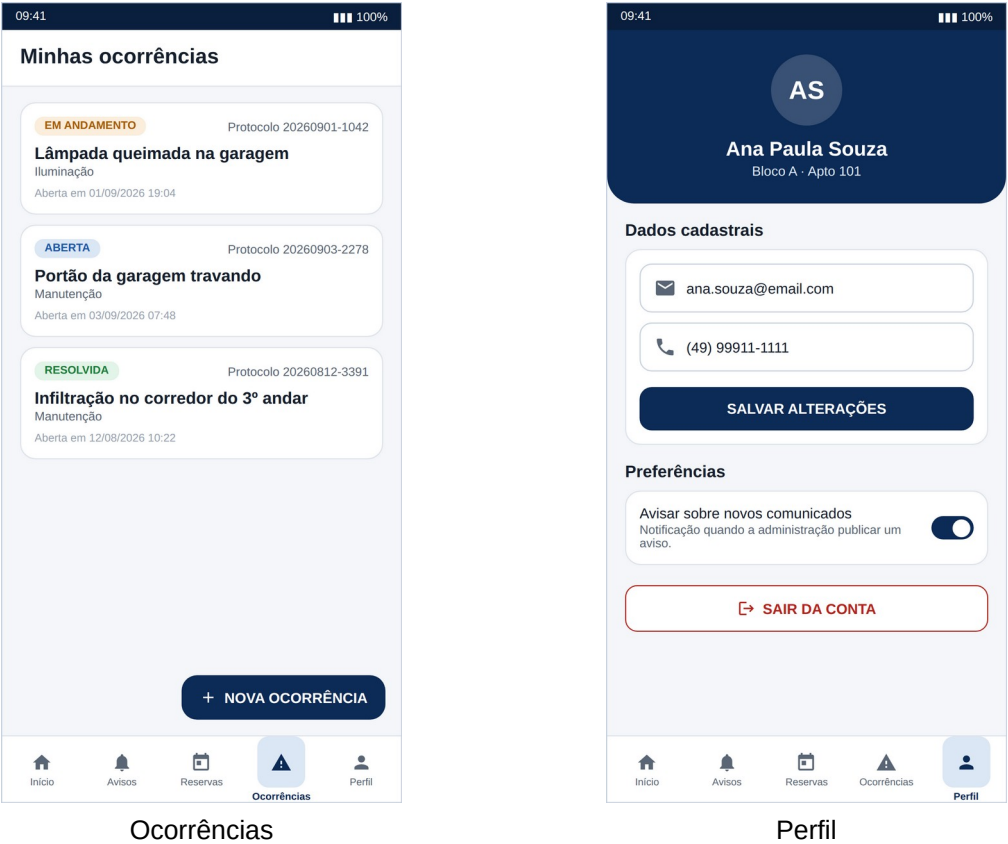
Nova reserva

Nova ocorrência

Fonte: o autor (2026).

A tela de nova reserva mostra os três passos ao mesmo tempo: área, data e horário. Os horários ocupados aparecem na grade, mas desabilitados e com a palavra "Ocupado". Preferi mostrá-los a escondê-los, porque sumir com opções dá a sensação de que o aplicativo perdeu informação. O botão de confirmar só habilita quando as três escolhas estão feitas.

Figura 7 - Lista de ocorrências e perfil



Fonte: o autor (2026).

Na lista de ocorrências, o protocolo aparece em todos os cards, porque é ele que o morador usa quando precisa cobrar a administração. No perfil, apenas e-mail e telefone são editáveis; bloco, apartamento e CPF ficam de fora porque são dados cadastrais, conforme a regra RN08.

5.5 Descrição das telas

Tabela 8 - Descrição das telas do protótipo

Tela	Objetivo	Principais elementos
Login	Identificar o morador e proteger o acesso.	Logotipo, campo de e-mail, campo de senha, botão Entrar, link de recuperação.
Início	Dar uma visão rápida da situação do morador no condomínio.	Saudação, unidade, contadores, próxima reserva, avisos recentes, atalhos.
Avisos	Centralizar os comunicados da administração.	Lista de cards com etiqueta de prioridade, título, data, resumo e autor.
Leitura do aviso	Apresentar o comunicado completo.	Etiqueta de prioridade, título, data, texto integral e autor.
Reservas	Mostrar e gerenciar as reservas do morador.	Lista com área, data, horário, etiqueta de status e ação de cancelar.
Nova reserva	Agendar uma área comum.	Seleção de área, seletor de data, grade de horários e botão de confirmar.
Ocorrências	Acompanhar os chamados abertos.	Lista com etiqueta de status, protocolo, assunto, categoria e data.
Nova ocorrência	Registrar um problema.	Seleção de categoria, campo de assunto com contador, campo de descrição e botão de registrar.
Perfil	Exibir os dados do morador e as preferências.	Avatar com iniciais, nome, unidade, e-mail, telefone, interruptor de notificação e sair.

Fonte: o autor (2026).

6 MVP, EVOLUÇÃO E RISCOS

6.1 MVP

A primeira versão entrega o conjunto mínimo que faz o aplicativo ser útil no dia a dia. São os requisitos marcados como essenciais na Tabela 1:

- Login com controle de acesso e sessão persistente.
- Tela inicial com o resumo do condomínio.
- Lista e leitura de comunicados.
- Reserva de área comum com verificação de disponibilidade e cancelamento.
- Registro e acompanhamento de ocorrências com protocolo.
- Perfil com dados de contato editáveis.

6.2 Evoluções previstas

Estas ficaram de fora da primeira versão por escopo, não por inviabilidade. A arquitetura em camadas comporta todas elas: cada uma entra como entidade, serviço e endpoint no servidor, e como uma área nova na navegação do aplicativo.

- Pagamento de taxas condominiais e segunda via de boleto.
- Repositório de documentos e atas.
- Assembleias e votações digitais.
- Conversa direta com a administração.
- Integração com a portaria e liberação de visitantes por QR Code.
- Painel administrativo dentro do próprio aplicativo.
- Notificação push com o aplicativo fechado.
- Versão para iOS consumindo a mesma API.

6.3 Riscos e mitigação

Tabela 9 - Riscos identificados

Risco	Impacto	Mitigação
Internet instável no momento do uso.	Ação não concluída e morador sem saber o que houve.	Mensagem clara distinguindo falta de rede de servidor fora do ar, com botão de tentar novamente.
Dois moradores reservam o mesmo horário.	Conflito de agenda e desgaste entre vizinhos.	Validação de sobreposição no servidor, no momento da gravação, com resposta específica para o segundo.

Risco	Impacto	Mitigação
Acesso indevido a dados de outro morador.	Exposição de informação pessoal.	Autenticação por token e endpoints que identificam o dono pela sessão, sem receber identificador por parâmetro.
Escopo maior do que o prazo comporta.	Entrega incompleta.	Separação explícita entre essencial e desejável, e desenvolvimento em etapas a partir do MVP.
Variedade de tamanhos de tela.	Interface cortada ou desproporcional.	Layouts com medidas relativas e testes em mais de uma resolução no emulador.
Senha fraca ou vazada.	Acesso indevido à conta do morador.	Armazenamento apenas do hash da senha e sessão com prazo de validade.

Fonte: o autor (2026).

7 CONCLUSÃO

O trabalho percorreu as etapas iniciais do projeto de um aplicativo mobile: partiu de uma necessidade real observada no condomínio onde moro, transformou essa necessidade em requisitos escritos e verificáveis, comparou as plataformas disponíveis e chegou a um protótipo de interface.

A escolha do Android nativo se sustenta em três razões que se reforçam: é onde está o público, é o que consigo desenvolver e testar com os recursos que tenho, e é o conjunto de ferramentas trabalhado na disciplina. A arquitetura em camadas foi desenhada de modo que essa escolha não vire uma amarra, já que a lógica de negócio fica no servidor e uma futura versão iOS consumiria a mesma API.

Os mockups mostram que as funcionalidades cabem em uma navegação curta. O morador entra, vê o que precisa saber na primeira tela, e alcança as duas ações mais comuns (reservar e reclamar) em um toque. Considero esse o principal resultado do protótipo: não é uma tela bonita isolada, é um conjunto em que cada decisão de layout responde a uma diretriz definida antes.

O que fica encaminhado para a próxima etapa é a implementação. Os requisitos funcionais essenciais formam a lista de tarefas, as regras de negócio da Tabela 3 são o que precisa ser testado com mais cuidado, e a arquitetura da Figura 1 diz onde cada pedaço de código deve morar.

REFERÊNCIAS

ANDROID DEVELOPERS. Guia de arquitetura de apps. Disponível em: <https://developer.android.com/topic/architecture>. Acesso em: ago. 2026.

ANDROID DEVELOPERS. Distribuição de versões da plataforma. Disponível em: <https://developer.android.com>. Acesso em: ago. 2026.

FOWLER, Martin. Padrões de arquitetura de aplicações corporativas. São Paulo: Novatec, 2006.

GOOGLE. Material Design 3: guidelines de componentes e acessibilidade. Disponível em: <https://m3.material.io>. Acesso em: ago. 2026.

MURRAY, Matt. Designing mobile interfaces. Sebastopol: O'Reilly Media, 2012.

PRESSMAN, Roger S. Engenharia de software: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

UNOESC. Desenvolvimento para dispositivos móveis. v. 2. Porto Alegre: SAGAH. Material didático da disciplina.

W3C. Web Content Accessibility Guidelines (WCAG) 2.1: contrast minimum. Disponível em: <https://www.w3.org/TR/WCAG21>. Acesso em: ago. 2026.